

10. Управление на паметта. Организация на разпределение на паметта при съвременни компютърни системи. Основни понятия и същност на виртуалната памет. Средства за преместване на процесите.

1. Управление на паметта

Управлението на паметта (Memory Management) е една от фундаменталните подсистеми на всяка съвременна операционна система (ОС). Нейната основна задача е да контролира и координира компютърната памет, като динамично заделя части от нея (блокове) на програмите при тяхното изпълнение и ги освобождава, когато вече не са необходими.

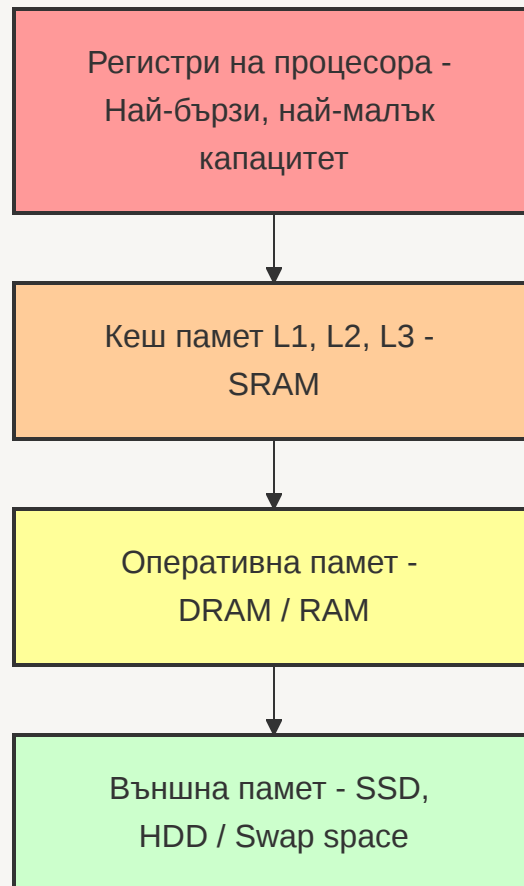
1.1 Цели и задачи на подсистемата за управление на паметта

- **Оптимално оползотворяване:** Максимизиране на броя на едновременно изпълняваните процеси (степен на мултипрограмиране).
- **Изоляция и защита:** Всеки процес трябва да оперира в собствено адресно пространство. Никой процес не трябва да има достъп (за четене или запис) до паметта на друг процес или на ядрото на ОС без изрично разрешение.

- **Споделяне на паметта:** Осигуряване на контролиран механизъм, чрез който два или повече процеса могат да споделят обща област от паметта (например за бърза междупроцесна комуникация - IPC).
- **Логическа и физическа организация:** Справяне с разминаването между линейната структура на физическата памет и модулната структура на потребителските програми.

1.2 Йерархия на паметта

В съвременните компютърни системи паметта е организирана в йерархична структура с цел балансиране на цената, капацитета и скоростта на достъп:



2. Организация на разпределение на паметта при съвременни компютърни системи

Разпределението на паметта определя по какъв начин физическата оперативна памет се предоставя на активните процеси. Исторически и архитектурно то се разделя на две основни категории: **непрекъснато** и **дискретно**.

2.1 Непрекъснато разпределение на паметта (Continuous Allocation)

При този подход всеки процес заема един единствен, непрекъснат блок от физически адреси.

Единична програма (Single-Partition Allocation)

Паметта се разделя на две части: една за ОС (обикновено в ниските адреси) и една за потребителския процес. Използва се в стари системи (напр. MS-DOS). Не поддържа мултипрограмиране.

Разпределение с фиксирани раздели (Static Partitioning)

Паметта се разделя на предварително дефинирани зони (статични раздели) с фиксиран размер (който може да е еднакъв или различен). Всеки раздел съдържа точно един процес.

Разпределение с динамични раздели (Dynamic Partitioning)

Разделите се създават динамично в процеса на работа, съобразно точните нужди на съответния процес. Първоначално паметта е един голям непрекъснат блок (hole). С навлизането и излизането на процеси паметта се фрагментира.

За разпределяне на процесите в свободните блокове се използват следните класически стратегии:

1. **First-Fit (Първо съвпадение):** Сканира се паметта от началото и се избира свободен блок, който е достатъчно голям. Стратегията е бърза.

2. **Best-Fit (Най-добро съвпадение):** Сканира се цялата памет и се избира най-малкият свободен блок, който удовлетворява заявката. Минимизира оставащото празно място в конкретния блок, но остава много малки неизползваеми дупки.
3. **Worst-Fit (Най-лошо съвпадение):** Избира се най-големият наличен свободен блок. Идеята е оставащата част от блока да бъде достатъчно голяма за друг процес, но на практика работи лошо.
4. **Проблем:** Външна фрагментация - съвкупността от всички свободни блокове в паметта е достатъчна за стартиране на нов процес, но те са разпръснати и няма нито един единичен непрекъснат блок с необходимия размер. Решава се чрез компактизиране (или съгъстяване на паметта), което е тежка операция за процесора.

2.2 Дискретно разпределение на паметта (Непоследователен Allocation)

Съвременните компютърни системи преодоляват външната фрагментация чрез дискретно разпределение, при което адресното пространство на даден процес се разделя на части, които могат да бъдат разположени на произволни, не непрекъснати места във физическата памет.

2.2.1 Странична организация на паметта (Paging)

Логическото адресно пространство на процеса се разделя на блокове с фиксиран размер, наречени **страници (pages)**. Физическата памет се разделя на блокове със същия размер, наречени **рамки (frames)**.

Връзката между логическите страници и физическите рамки се поддържа от структура от данни, наречена **Таблица на страниците (Page Table)** за всеки процес.



2.2.2 Сегментна организация на паметта (Segmentation)

Логическото адресно пространство се разглежда като съвкупност от логически единици с променлива дължина, отразяващи структурата на програмата (функция main, обекти, стек, масиви). Тези единици се наричат **сегменти**. Всеки сегмент има име (или номер) и дължина. Преводът на адреси става чрез **Таблица на сегментите (Segment Table)**, съдържаща базов адрес във физическата памет и лимит (дължина) на сегмента.

Характеристика	Странична организация (Paging)	Сегментна организация (Segmentation)
Размер на блоковете	Фиксиран (обикновено 4 KB).	Променлив (динамичен).
Фрагментация	Вътрешна фрагментация (в последната страница).	Външна фрагментация (между сегментите).
Видимост за програмиста	Напълно прозрачна (невидима).	Видима (програмата се разделя на логически модули).

2.2.3 Сегментно-странична организация

Комбинираща предимствата на двата метода. Адресното пространство се разделя логически на сегменти, а всеки сегмент от своя страна се разделя на страници с фиксиран размер. Това е стандартният подход при съвременните x86/x64 архитектури.

3. Основни понятия и същност на виртуалната памет

Виртуалната памет (Virtual Memory) е концепция и архитектурна техника, която позволява на процесите да се изпълняват, дори когато тяхното адресно пространство надхвърля физически наличната оперативна памет (RAM).

3.1 Същност и разделяне на адресните пространства

Виртуалната памет разделя паметта на 2 понятия:

1. **Логическо адресно пространство:** Множеството от адреси, които даден процес генерира по време на изпълнението си.

2. **Физическо адресни пространство:** Множеството от реалните адреси, съществуващи в хардуерните чипове на оперативната памет.

Преводът от виртуални към физически адреси се извършва в реално време от специализиран хардуерен модул в процесора, наречен **MMU (Memory Management Unit)**.

3.2 Странициране по заявка (Demand Paging)

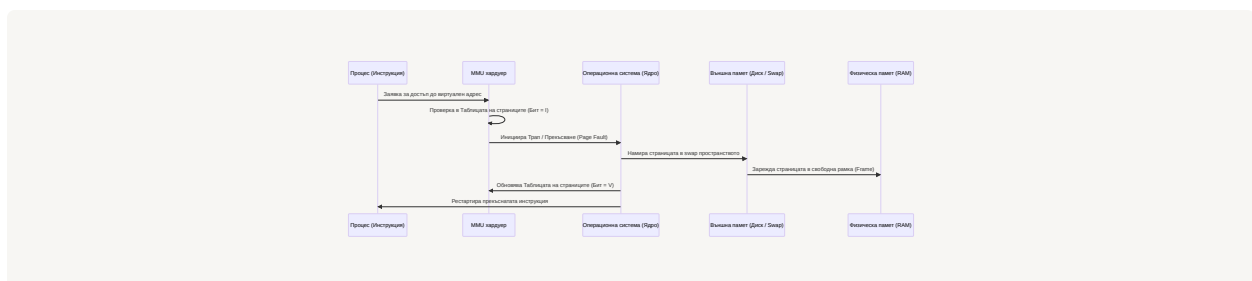
Повечето съвременни системи имплементират виртуалната памет чрез механизма странициране по заявка. Вместо цялата програма да се зарежда в RAM при стартиране, страниците се зареждат в паметта само когато са необходими за изпълнението на конкретната инструкция.

За целта във всеки запис на Таблицата на страниците (Page Table Entry) се поддържа т.нар. бит за валидност/присъствие (Valid/Invalid bit):

- **V(Valid):** Страницата се намира във физическата памет (RAM)
- **I(Invalid):** Страницата не е валидна или се намира на твърдия диск (в т.нар. swap space)

3.3 Механизъм на "Грешка в страницата" (Page Fault)

Когато процесът се опита да достъпи виртуален адрес, чиято страница е маркирана като Invalid, хардуерът (MMU) генерира прекъсване към процесора, известно като **Page Fault (Грешка в страницата)**.



3.4 Структури за ускоряване на транслацията

Тъй като таблиците на страниците могат да бъдат огромни и се съхраняват в RAM, всеки достъп до паметта би изисквал двойно време (веднъж за

таблицата, веднъж за данните). За целта се използват:

- **TLB(Translation Lookaside Buffer):** Малък асоциативен, високоскоростен хардуерен кеш вътре в самия процесор. Той съхранява адресите на последно използваните страници. При TLB hit преводът е почти мигновен, При TLB miss се чете от главната таблица в RAM.
- **Инвертирана таблица на страниците (Inverted Page Table):** Вместо по една таблица за всеки процес, в системата има само една глобална таблица, съдържаща по един запис за всяка реална физическа рамка. Намалява драстично консумацията на памет при системи с много процеси.

3.5 Алгоритми за заместване на страници (Page Replacement Algorithms)

Когато възникне Page Fault, а във физическата памет няма свободна рамка, ОС трябва да избере страница, която да бъде изхвърлена на диска (victim page). Ефективността на виртуалната памет зависи критично от избора на алгоритъм за заместване, за да се избегне състояние, в което системата прекарва повече време в четене/запис на страници от диска, отколкото в изпълнение на реална работа.

1. FIFO (First In - First Out): Заменя се страницата, която е престояла най-дълго в паметта.
2. LRU (Least Recently Used): Заменя се страницата, която не е била достъпвана най-дълго време назад в миналото. Изисква сериозна поддръжка за проследяване на времето.
3. Оптимален алгоритъм (MIN / OPT): Заменя се страницата, която няма да бъде използвана за най-дълъг период от време в бъдещето. Той е теоретичен и недостижим на практика, тъй като изисква бъдещо знание за поведението на програмата. Използва се като еталон за сравнение.

4. Средства за преместване на процесите (Process Relocation)

В мултипрограмните среди процесите непрекъснато се зареждат, изпълняват, блокират или прекъсват. За оптималното използване на ресурсите е необходимо процесите да могат да променят физическото си местоположение в паметта - концепция, известна като **преместване (relocation)**.

4.1 Свързване на адреси (Address Binding)

Преди една програма да се изпълни, нейните символни адреси (имена на променливи, етикети на функции) трябва да бъдат обвързани с конкретни адреси в паметта. Този процес се случва в три възможни етапа:

1. **Време на компилация (Compile Time):** Ако местоположението на процеса в паметта е известно предварително, се генерира абсолютен код. Ако началният адрес се промени, кодът трябва да се компилира наново.
2. **Време на зареждане (Load Time):** Компиляторът генерира код, който може да се премества. Обвързването се извършва в момента, в който програмата се зарежда от диска в оперативната памет от специален системен модул (Loader). Физическите адреси остават фиксирани по време на целия цикъл на изпълнение.
3. **Време на изпълнение (Execution / Run Time):** Свързването се отлага до самия момент на достъп до инструкцията по време на работа. Ако процесът се премести в друг сегмент на паметта, обвързването се актуализира динамично. **Това е стандартът при съвременните системи.**

4.2 Динамично преместване и хардуерни средства

Динамичното преместване по време на изпълнение изисква директна хардуерна поддръжка от MMU. Най-простият и класически хардуерен механизъм включва двойка регистри:

- **Базов регистър:** Съдържа началния физически адрес на процеса в оперативната памет.
- **Лимитен регистър:** Съдържа логическия размер на адресното пространство на процеса (за целите на защитата).

Всяка адресна трансляция преминава през следната проверка и трансформация:

Ако Логически адрес < Лимитен регистър

Тогава Физически адрес = Логически адрес + Базов регистър

Ако логическият адрес надхвърли лимитния регистър, хардуерът генерира изключение за нарушена сигурност на паметта (Segmentation Fault / Trap).

Пример:

- Базов регистър (Base Register) = 10 000 (Това е реалният физически адрес в RAM паметта, от който започва паметта на процеса).
- Лимитен регистър (Limit Register) = 5 000 (Това е размерът на заделеното адресно пространство за процеса в байтове).

Това означава, че процесът има право да достъпва физически адреси в диапазона от 10 000 до 14 999 .

Да приемем, че процесорът (CPU) изпълнява инструкция и генерира Логически адрес: 345 . MMU извършва следните 2 стъпки:

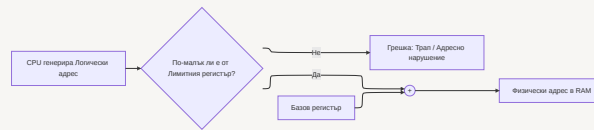
1. Проверка за сигурност

- а. Формула: Логически адрес < Лимитен регистър
- б. Проверка: 345 < 5 000
- в. Резултат: Да

2. Транслация:

- а. Формула: Физически адрес = Логически адрес + Базов регистър
- б. Изчисление: 345 + 10 000 = 10 345

Краен резултат: Процесорът успешно осъществява достъп до клетка с реален физически адрес 10 345 в чипа на оперативната памет (RAM).



4.3 Swapping

Swapping е мощно софтуерно-хардуерно средство за управление на процесите, което позволява даден процес да бъде временно изваден изцяло или частично от оперативната памет записан във вторичната външна памет (swar space). По-късно процесът се връща обратно в RAM за продължаване на изпълнението.

- При статично или Load time обвързване: Процесът трябва да се върне на точното физическо място, откъдето е изваден.
- При динамично Run-time обвързване: Процесът може да бъде върнат в съвсем различна зона на физическата памет, тъй като базовият регистър в MMU просто ще се презареди с новия начален физически адрес. Това позволява ефективна дефрагментация на паметта чрез уплътняване.

При съвременните ОС класическото swapping на цели процеси се среща рядко - то е заменено от **страничен swapping (page swapping)**, при което се преместват само отделните виртуални страници, а не целите процеси.